

Объектно- Ориентированное Программирование

(МИСиС, зима-весна 2026)

Полевой Дмитрий Валерьевич

к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

Хранение структурированных данных

- объемы небольшие (текстовые форматы)
 - TOML, JSON/YAML, CSV/TSV и т.п.
- объемы большие (бинарные форматы)
 - изображения/видео
 - HDF5
 - БД (SQLite)
 - и т.п.

TOML

- человекочитаемый, похож на INI
- однозначно соответствует ассоциативному массиву
- основан на
 - парах ключ = "значение«
 - [разделах]
 - # комментариях

TOML

(пример)

Документ TOML.

title = "Пример TOML"

[servers] *# отступы допустимы*

[servers.alpha]

ip = "10.0.0.1"

dc = "eqdc10"

Иерархические «документы»

- данные структурированы иерархически
- текстово-теговые форматы
 - теги, атрибуты,
- примеры форматов
 - HTML, XML, JSON

Примеры данных

формат XML

```
<?xml version="1.0"?>
<persons>
  <person username="MP123456">
    <name>Иван</name>
    <surname>ИВАНОВ</surname>
  </person>
</persons>
```

формат JSON

```
{
  "persons": [
    {
      "username": "MP123456",
      "name": "Иван ",
      " surname": "ИВАНОВ "
    }
  ]
}
```

Открытые библиотеки

- Nlohmann/json
- RapidJSON
- SimpleJSON
- sajson

и т.д.

Document Object Model (DOM)

(объектная модель представления)

- API манипуляции иерархическими данными
- все узлы дерева размещаются в памяти
- документ можно изменять (структура, данные)
- существуют различные способы
 - навигация «курсором» по дереву
 - процессоры запросов
 - процессоры схем валидации и трансформации

Simple API for XML (SAX)

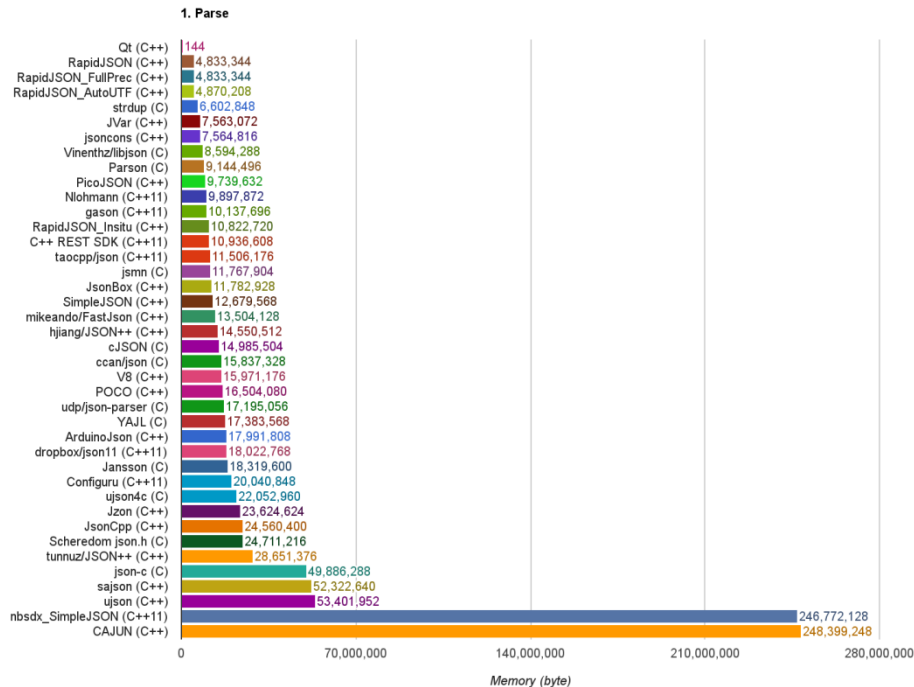
(событийный парсер)

- API чтения иерархических форматов
- требует фиксированного количества памяти
- не позволяет изменять содержимое документа
- сообщает посредством функций обратного вызова
 - распознанные элементы разметки
 - встретившиеся ошибки

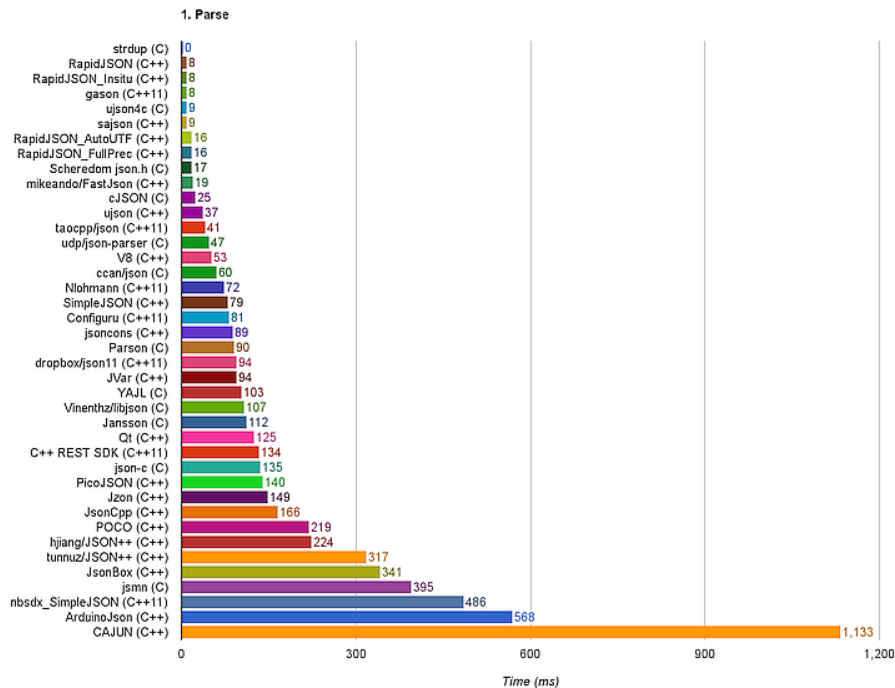
Как выбрать библиотеку?

- функционал
- лицензия
- живость проекта (активность разработчиков)
- возможность собрать (стандарт, скрипты сборки)
- хар-ки (потребляемая память, скорость работы)
- удобство API

Потребление памяти (json-парсеры)



Время разбора (json-парсеры)



ваши вопросы

Зачем что-то измерять «про время»?

- производительность – важна для «заказчика»
- укладываться в ограничения
- сравнивать/выбирать «разные версии»
- прогнозировать «а что будет если...»
- выявлять «узкие места»

Как замерять время?

`<chrono>`

- современная библиотека (разобраться)

`<ctime>`

- устаревшая C-библиотека
- низкая точность (большие интервалы)

Основные понятия

в пространстве имен `std::chrono`

- `clock` - таймер
- `time_point` - моменты времени
- `duration` - интервал (разность моментов)

Таймеры

- `system_clock`
 - системный (может быть изменен)
 - обычно, с 01.01.1970
- `steady_clock` – устойчивый
- `high_resolution_clock` – наиболее точный

Схема замера

```
auto t_beg = steady_clock::now();  
// do some job  
auto t_end = steady_clock::now();  
auto elapsed = duration_cast<milliseconds>  
    (t_end - t_beg);
```

Результаты измерения

- зависят от флагов
 - различать Debug и Release
 - понимать, для какого набора измеряете
- изменяются
 - зависят от «загруженности» компьютера
 - зависят от железа/ОС/системы сборки

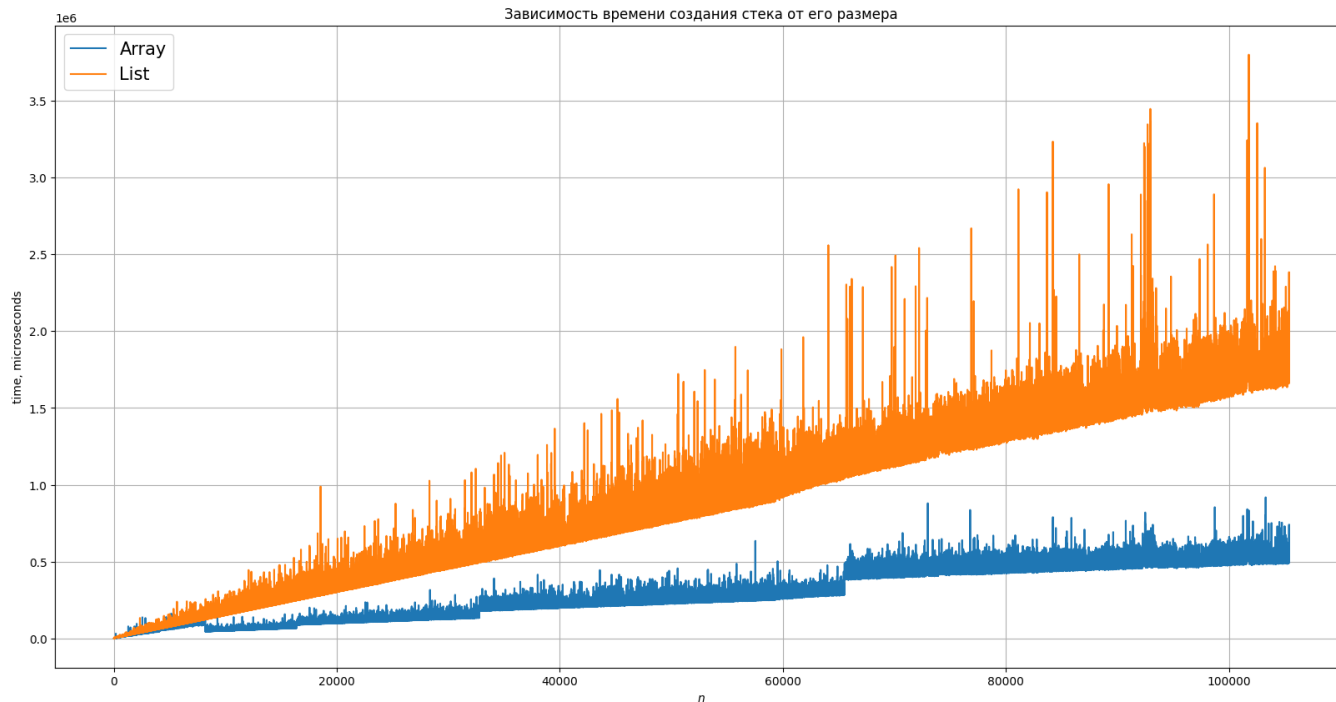
Результаты измерения (практические рекомендации)

- абсолютные числа часто не информативны
- анализировать относительные показатели
 - понимать, для какого набора операций
- повторять достаточное количество раз
- разобраться со статистической обработкой

Информативные формы представления результатов

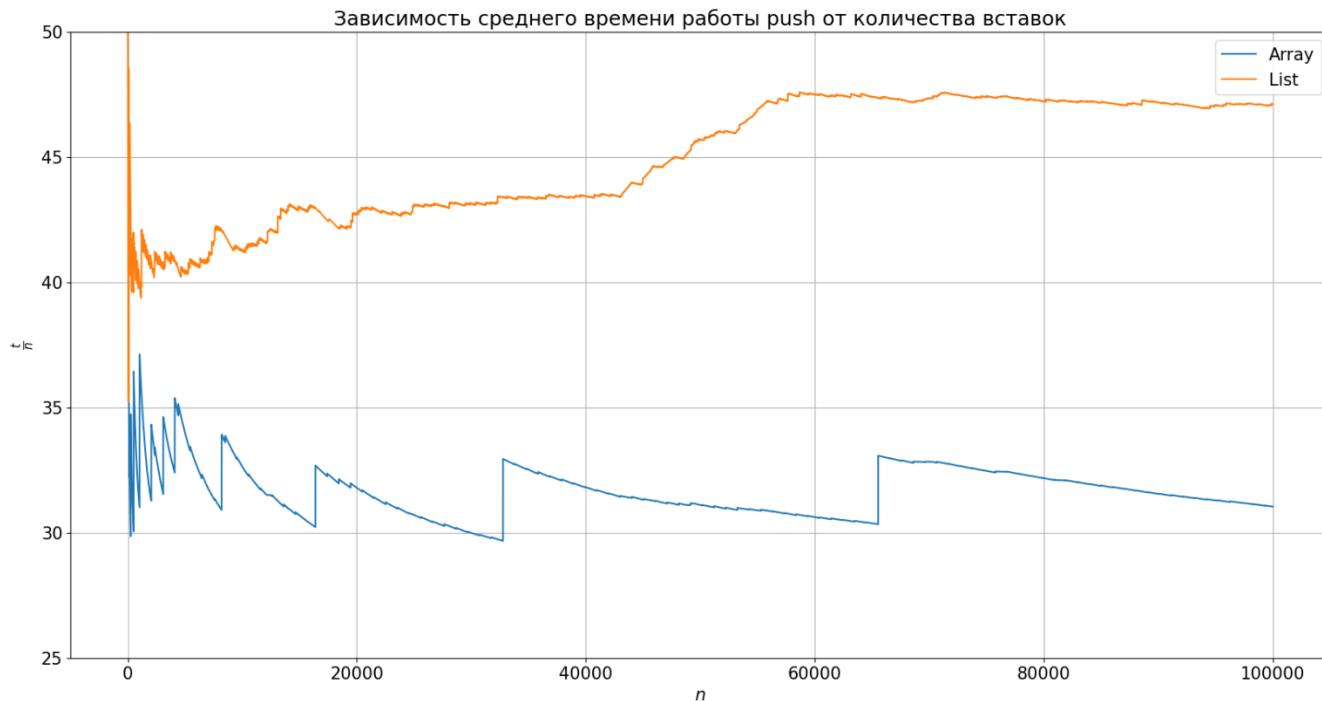
- текст
- таблицы
- графики
- ЧТО хотите показать
- КАК передать максимально точно

График (общая тенденция)



График

(общая тенденция)



Графики

(на что обращать внимание)

- ищите/анализируйте отличные/плохие образцы
- «послание» должно читаться четко/однозначно
- типы отображения
- выбор показателей (для осей)
- подписи величин и отметки (для осей)
- масштабы, полнота использования

ваши вопросы

Что такое 'warning'?

Предупреждения (warnings)

- диагностические сообщения при сборке
 - часто показывают потенциально опасные места
 - чаще от компилятора, реже от линковщика
 - требуют внимания
-
- зависят от среды и настроек сборки

Предупреждения (примеры MS VC 2022)

C6001: Using uninitialized memory 'variable'

C6011: Dereferencing NULL pointer '*pointer-name*'

C6014: Leaking memory 'pointer-name'.

C6029: Possible buffer overrun in call to 'function'

C6101: Returning uninitialized memory 'parameter-name'

LNK4006:

symbol already defined in object; second definition ignored

Предупреждения (некоторые флаги MS VC)

/W0 - подавить все предупреждения

/W1 - тяжелые

/W2 - значимые (уровень 1-2)

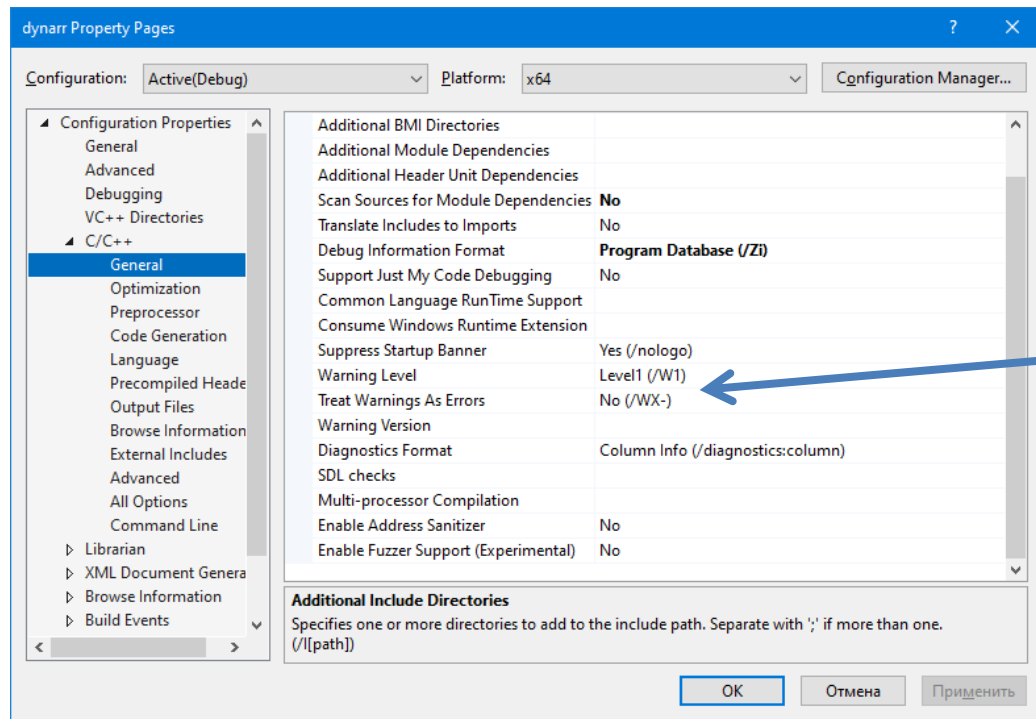
/W3 - качество рабочей среды (уровень 1-3)

/W4 - информационные (уровень 1-3)

/Wall - показать все

/WX - предупреждения компилятора как ошибки

Предупреждения (в настройках проекта MS VC)



Warning Level	Level1 (/W1)
Treat Warnings As Errors	No (/WX-)
Warning Version	

Предупреждения (некоторые флаги gcc)

- w - подавить все предупреждения
- Wpedantic - строгий C++
- Wall - все неоднозначные конструкции
- Wextra - и ещё немного больше сообщений
- Werror - предупреждения как ошибки

Предупреждения (рекомендации)

- следите за появлением и устраняйте
- исправляйте код вдумчиво, а не механически
- найдите правильный способ «подавить»
 - cross-platform (обычно через макросы)
 - если «не ваши» = «не исправить»
 - если «ваши», но «так надо»

Предупреждения (рекомендации)

- разные для разных систем сборки
- при пересборке выводятся не все
- явное управление степенью «параноидальности»
- крайний вариант Warning As Error
- сообщения от моей системы сборки для вас
`/prj.logs/*.msvc.msbuild.warnings.log`

Предупреждения

(пример подавления в сторонних библиотеках)

```
#include <lib/lib.hpp>
```

```
#define DOCTEST_CONFIG_IMPLEMENT_WITH_MAIN
```

```
#include <doctest/doctest.h>
```

```
DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_BEGIN
```

```
#include <iostream>
```

```
#include <string>
```

```
#include <vector>
```

```
DOCTEST_MAKE_STD_HEADERS_CLEAN_FROM_WARNINGS_ON_WALL_END
```

ваши вопросы

Бинарный интерфейс приложений

(Application Binary Interface, ABI)

- интерфейс между двумя бинарными программными компонентами (часто библиотеки и/или компоненты ОС)
- при соблюдении позволяет компоновщику объединять откомпилированные модули без перекомпиляции всего исходного текста

Бинарный интерфейс приложений

(много-много технических деталей...)

- использование регистров процессора
- состав и формат системных вызовов и вызовов одного модуля другим
- формат передачи аргументов и возвращаемого значения при вызове функции
- размеры типов, выравнивания, исключения
- и т.д.

Согласованность ABI

- достигается настройками (сборки)
- бинарные компоненты
 - могут использоваться повторно/долго
 - могут быть вызваны из других языков

C-интерфейс

- C-интерфейс – универсальное соглашение
 - типы, соглашения вызова, имена
- проставить макросы в настройках проектов
 - CMake скрипты
 - исходные файлы
 - IDE

Подавление декорирования имен

```
#ifdef __cplusplus
extern "C" {
#endif //__cplusplus

// функции C-интерфейса

#ifdef __cplusplus
} // end of extern "C" {
#endif //__cplusplus
```


Макросы для экспортируемой С-функции

```
#ifdef _WINDOWS
#    define DLL_CALL __cdecl
#else
#    define DLL_CALL
#endif

#ifdef LIBNAME_EXPORTS
#    define DLL_EXPORT __declspec(dllexport)
#else
#    define DLL_EXPORT
#endif
```

Пример экспортируемой С-функции

```
// in .h
```

```
DLL_EXPORT int DLL_CALL st(const int n);
```

```
// in .cpp
```

```
int st(const int n) {  
    // ...  
}
```

Как использовать динамически библиотеки

- загружать/выгружать руками
 - дублирование кода
 - утечки ресурсов
- использовать библиотеку-обертку
 - dylib (как пример)

dylib

```
// Default OS decorations added  
// Windows -> "foo.dll"  
// MacOS    -> "libfoo.dylib"  
// Linux    -> "libfoo.so"
```

```
dylib::library lib("./foo",  
    dylib::decorations::os_default());
```

dylib - C символы

```
auto adder = lib.get_function<double(double,  
    double)>("adder");  
double pi =  
    lib.get_variable<double>("pi_value");  
  
double result = adder(pi, pi);
```

dylib – C++ символы

```
auto int_to_string =  
    lib.get_function<std::string(int)>(  
        "tools::to_string(int)");  
  
auto double_to_string =  
    lib.get_function<std::string(double)>(  
        "tools::to_string(double)");
```

SWIG

(Simplified Wrapper and Interface Generator)

- кодогенератор для связывания C/C++ с
 - Python, Ruby, PHP, Tcl, Perl
 - Java, C#, Scheme, OCaml
- по файлам с описанием экспортируемых функций
- генерируется «клеевой» код

ваши вопросы

Что ломает/изменяет ABI

- изменение состава полей
- изменение шаблонных параметров
- превращение шаблонной функции в нешаблонную или наоборот
- добавление `variadic template`

Что ломает/изменяет ABI

- изменение спецификаторов `inline`
 - изменение параметров по умолчанию
 - объявление нового виртуального метода
- и т.д.

Pointer to Implimentation

(PImpl)

альтернативные названия

- d-pointer
- Bridge
- Cheshire Cat
- Handle/Body
- Compilation Firewall

PIML

(за что борьба)

```
// в заголовке w.hpp  
class W {  
public:  
    W();  
private:  
    std::string name_;  
    std::vector<double> data_;  
};
```

PIML

(за что борьба)

- для компиляции типа W требуется включить заголовки `<string>`, `<vector>`, и т.д.
- заголовки увеличивают время сборки клиентов типа W (включающих `w.hpp`)
- при изменении `w.hpp` $\rightarrow$ перекompиляция

PIMPL

(идея)

- в заголовке оставить только объявление (стабильного) интерфейса
- реализацию сделать в отдельном типе, который в заголовке только объявить
- объявить член-указатель на тип реализации
- определение типа реализации сделать в файле реализации

PIMPL

(результаты)

- тип реализации можно изменять без перекомпиляции остального кода (стабильность)
- сокрытие имен из области видимости
- ускорение сборки/пересборки
- лишний уровень косвенности (вызов через указатель)

PIMPL

(cxema - w.hpp)

```
#include <memory>

class W {
    // ...
private :
    struct Impl;
    std::unique_ptr<Impl> p_;
};
```


PIMPL

(cxema - w.hpp)

```
public:  
    W();  
    ~W();  
    W(const W& src);  
    W(W&& src);  
    W& operator=(const W& src);  
    W& operator=(W&& src);
```

PIMPL

(cxema - w.cpp)

```
#include "w.hpp"
#include <string>
#include <vector>

struct W::Impl {
    std::string name_;
    std::vector<double> data_;
};
```

PIMPL

(cxema - w.cpp)

```
W::W()  
    : p_(std::make_unique<Impl>()) {  
}
```

```
W::~~W() = default;
```

```
W(W&& src) = default;
```

```
W& operator=(W&& src) = default;
```

PIMPL

(cxema - w.cpp)

```
W::W(const W& src) {  
    if (src.p_) {  
        p_ = std::make_unique<Impl>(*src.p_);  
    }  
}
```

PIMPL

(cxema - w.cpp)

```
W& W::operator=(const W& src) {  
    if (this != &src) {  
        // ..  
    }  
    return *this;  
}
```

PIMPL

(cxema - w.cpp)

```
if (src.p_) {  
    if (p_) {  
        *p_ = *src.p_;  
    } else {  
        p_ = std::make_unique<Impl>(*src.p_)  
    } else {  
        p_.reset();  
    }  
}
```

ВАШИ ВОПРОСЫ

TEST_CASE_TEMPLATE

- шаблонный тест-кейс для заданного набора типов
- изменение набора типов не удобно (`#define`)

```
TEST_CASE_TEMPLATE("id", T, char, long) {  
    T var = T{};  
    --var;  
    CHECK(var == -1);  
}
```


TEST_CASE_TEMPLATE

(для тренировки и проверки)

```
#define TESTED_TYPES char, long

TEST_CASE_TEMPLATE("id", T, TESTED_TYPES) {
    T var = T{};
    --var;
    CHECK(var == -1);
}
```

TEST_CASE_TEMPLATE_DEFINE

- шаблонный тест для будущего инстанцирования

```
TEST_CASE_TEMPLATE_DEFINE("id", T, test_id) {  
    T var = T();  
    --var;  
    CHECK(var == -1);  
}
```

TEST_CASE_TEMPLATE...

- инстанцирование шаблонного теста

```
TEST_CASE_TEMPLATE(INVOKE(test_id,  
    char, short, int, long long int);
```

```
TEST_CASE_TEMPLATE(APPLY(test_id,  
    std::tuple<float, double>);
```

Шаблонные тесты doctest

- автор библиотеки создает шаблонные тесты
- авторы реализаций инстанцируют тесты для СВОИХ ТИПОВ
- «зависящие от типа» фрагменты теста можно выносить в шаблонные функции
- делать специализации для специфичных типов

Шаблонная проверка LIFO

(подготовка данных)

```
StackLstT<T> st;  
std::vector<T> data;  
GetD(40, data);  
for (int i = 0; i < std::ssize(data); i += 1) {  
    st.Push(data[i]);  
}
```

Шаблонная проверка LIFO (проверка порядка данных)

```
bool correct_lifo = true;
for (int i = std::ssize(data) - 1;
     correct_lifo && 0 <= i; i -= 1) {
    correct_lifo = data[i] == st.Top();
    st.Pop();
}
CHECK(correct_lifo);
```

Шаблонная проверка

(универсальное заполнение элементами)

```
template<class T>
void GetD(int n, std::vector<T>& d) {
    d.clear();
    d.reserve(n);
    for (int i = 0; i < n; i += 1) {
        d.emplace_back(i);
    }
}
```

Шаблонная проверка

(специализированное заполнение элементами)

```
template<>
void GetD(int n, std::vector<std::string>& d) {
    d.clear();
    d.reserve(n);
    for (int i = 0; i < n; i += 1) {
        d.emplace_back(std::to_string(i));
    }
}
```


ваши вопросы